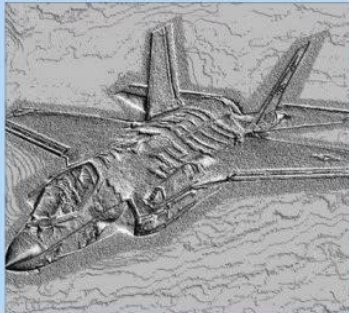
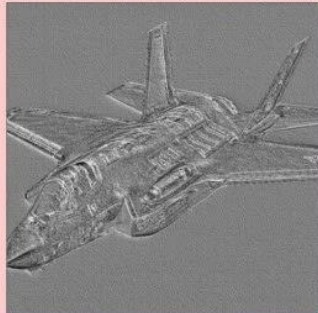


How to detect AI-generated images?

Real

Fake

Luminance

$$L(x, y) = 0.2126 R(x, y) + 0.7152 G(x, y) + 0.0722 B(x, y)$$

Gradients

$$G_x(x, y) = \frac{\partial L(x, y)}{\partial x}, G_y(x, y) = \frac{\partial L(x, y)}{\partial y} \Rightarrow \mathbf{v}(x, y) = \begin{bmatrix} G_x(x, y) \\ G_y(x, y) \end{bmatrix}$$

Flatten

$$\mathbf{M} = \begin{bmatrix} G_x(1) & G_y(1) \\ G_x(2) & G_y(2) \\ \vdots & \vdots \\ G_x(N) & G_y(N) \end{bmatrix}, \mathbf{M} \in \mathbb{R}^{N \times 2} \Rightarrow \mathbf{C} = \frac{1}{N} \mathbf{M}^T \mathbf{M}$$

2D vector per pixel

Covariance Matrix



Carlos Gabriel ... • 3er+
Administrador en rogerbit.com
1 día •

+ Seguir

✕

La técnica sencilla que diferencia imágenes reales de las generadas por IA

Un método sorprendentemente simple está demostrando ser eficaz para detectar si una imagen es real o fue creada por un modelo de difusión: analizar sus campos de gradiente y proyectarlos con PCA.

Las fotografías capturadas por cámaras mantienen gradientes coherentes, influenciados por la óptica, la iluminación y el sensor. En cambio, las imágenes generadas por IA presentan patrones de alta frecuencia menos estables, resultado del proceso de eliminación de ruido característico de los modelos de difusión.

El procedimiento consiste en convertir la imagen a luminancia, calcular sus gradientes en las direcciones horizontal y vertical, construir una matriz de covarianza con esos datos y aplicar PCA. Al comparar diferentes imágenes, las proyecciones resultantes tienden a separarse de manera consistente entre reales y sintéticas.

No es un método infalible ni reemplaza análisis forense avanzados, pero ofrece una herramienta ligera, rápida y explicable para identificar diferencias estructurales invisibles al ojo humano. Una forma práctica de explorar la autenticidad de una imagen en una época donde lo artificial y lo real se parecen cada vez más.

510

15 comentarios · 26 veces compartido

Recomendar

Comentar

Compartir

Enviar

Añadir un comentario...

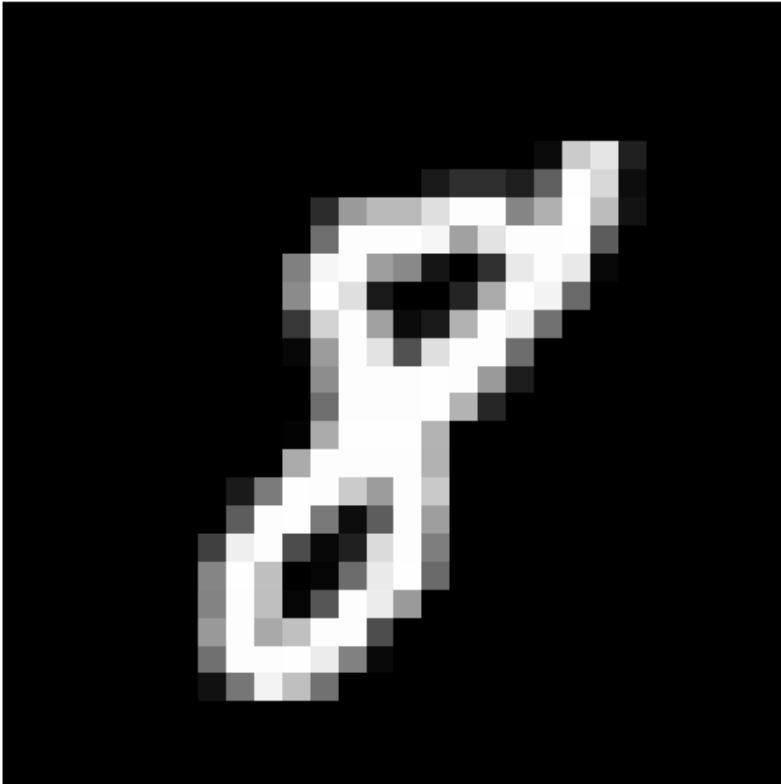
Can we recognize the input image after applying pooling?

- As I said the last day, in principle, yes!
- But, as usually, it depends...
“the devil is in the details”

Can we recognize the input image after applying pooling?

- MNIST

Imagen de Entrada Original. size=(28,28)



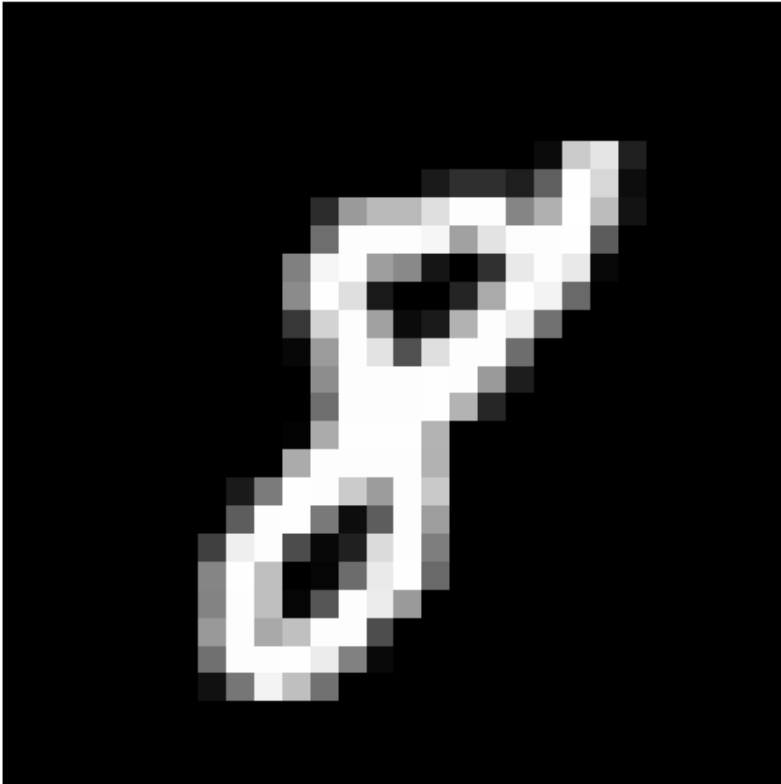
Max Pooling (3x3, stride 3)

Average Pooling (3x3, stride 3)

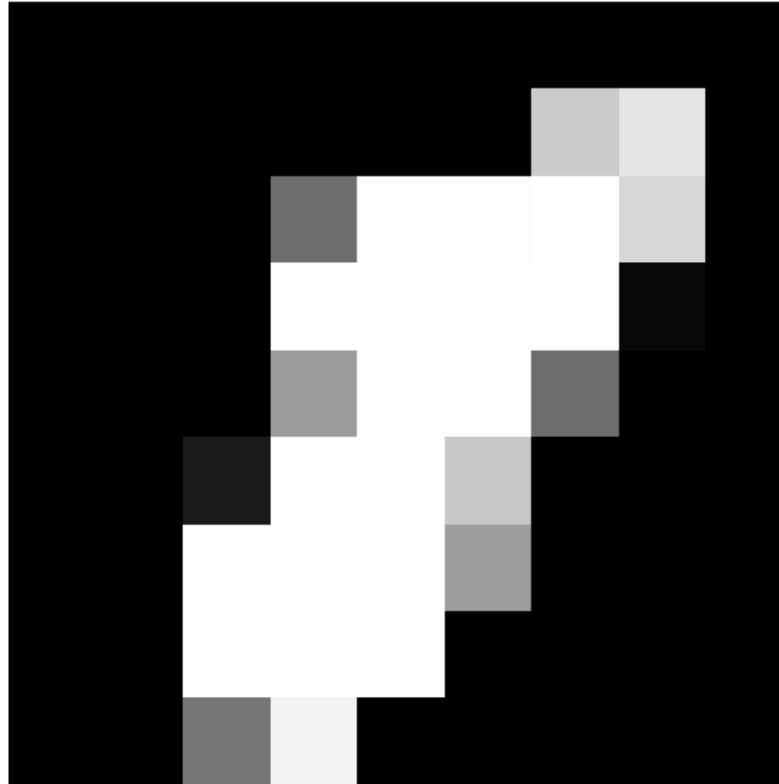
Can we recognize the input image after applying pooling?

- MNIST

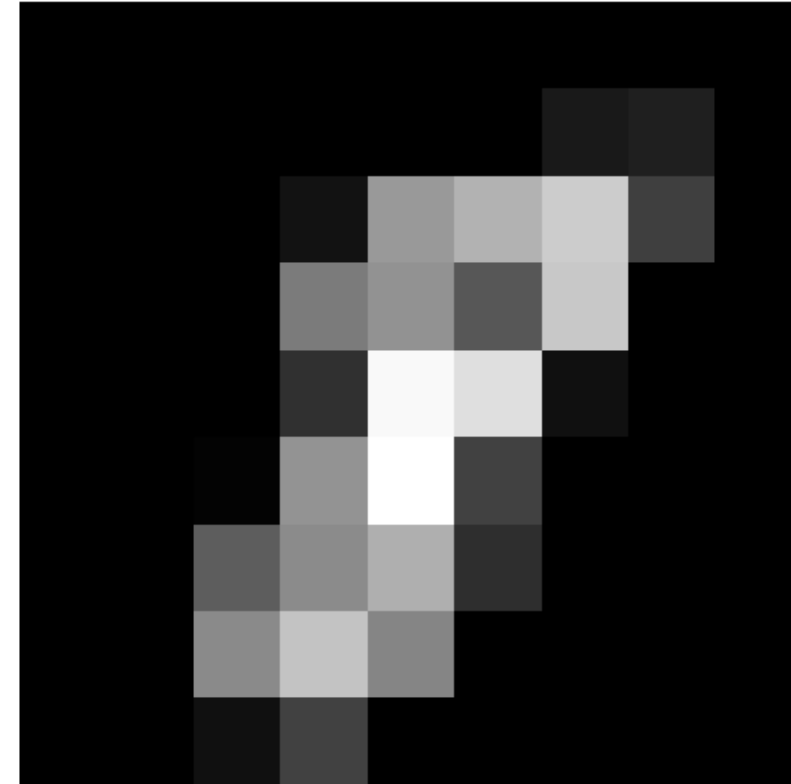
Imagen de Entrada Original. size=(28,28)



Max Pooling (3x3, stride 3)



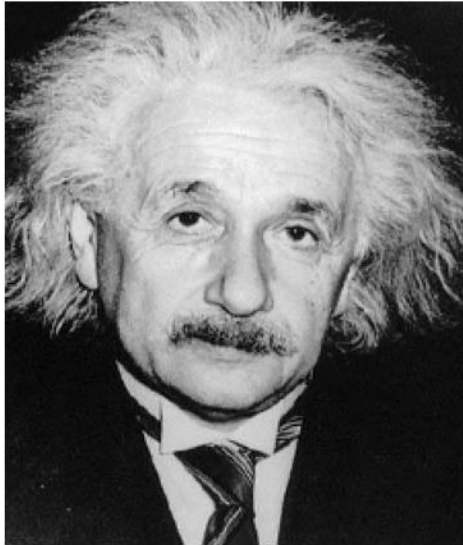
Average Pooling (3x3, stride 3)



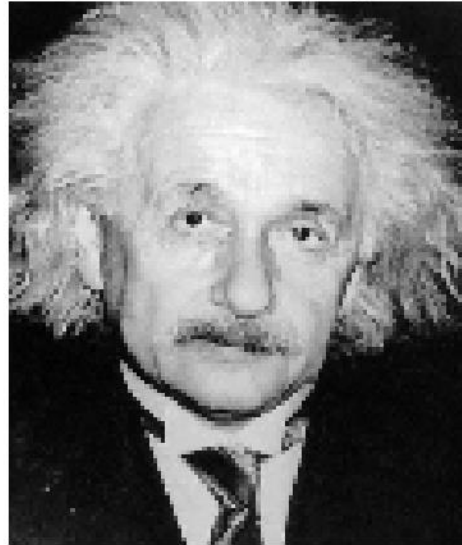
It's like filtering. **Everything depends on the filtering performed.** For instance, if **excessive smoothing is applied**, the input image also becomes **unrecognizable/unidentifiable**.

Can we recognize the input image after applying pooling?

Imagen de Entrada Original. size=(265,225)



Max Pooling (2x2, stride 2)



Average Pooling (2x2, stride 2)

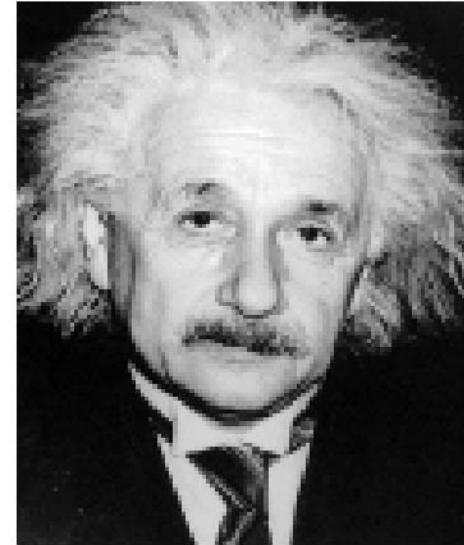
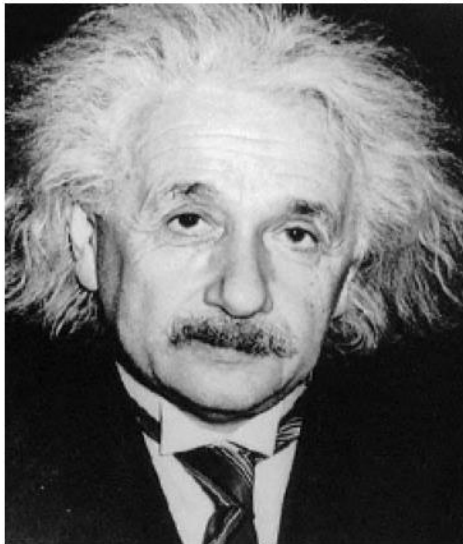
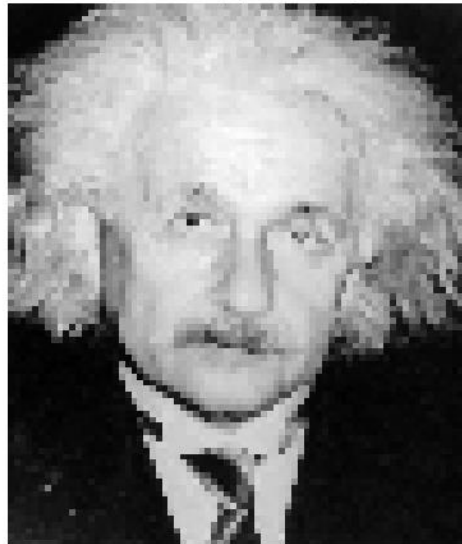


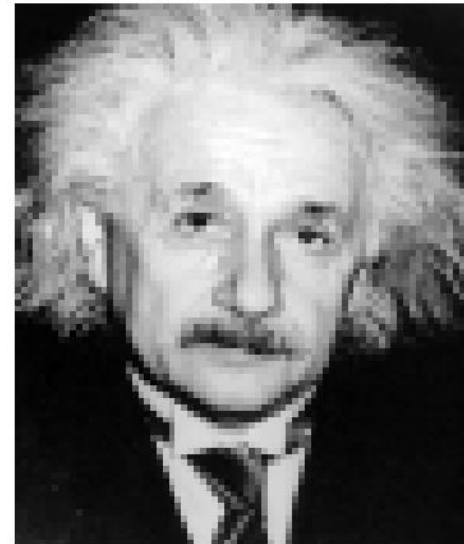
Imagen de Entrada Original. size=(265,225)



Max Pooling (3x3, stride 3)

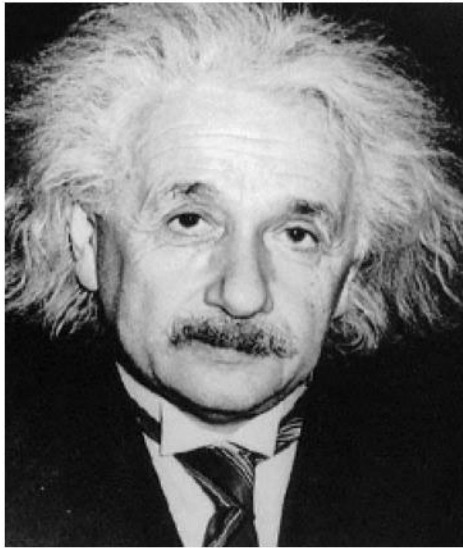


Average Pooling (3x3, stride 3)



Can we recognize the input image after applying pooling?

Imagen de Entrada Original. size=(265,225)



Max Pooling (5x5, stride 5)



Average Pooling (5x5, stride 5)

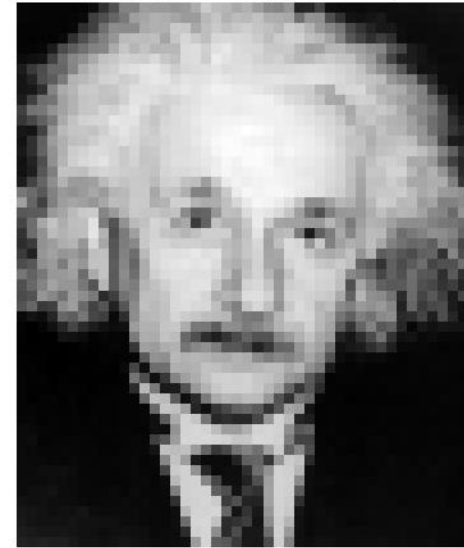
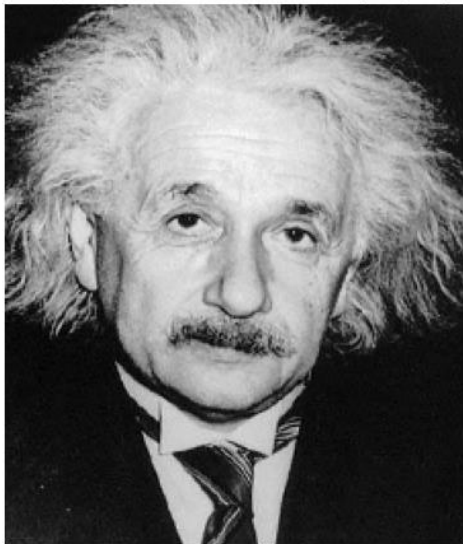


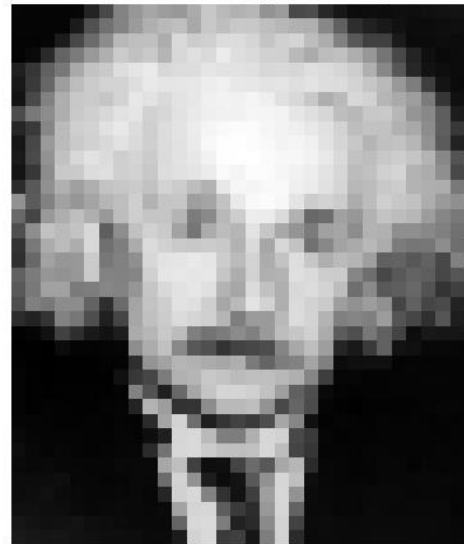
Imagen de Entrada Original. size=(265,225)



Max Pooling (7x7, stride 7)

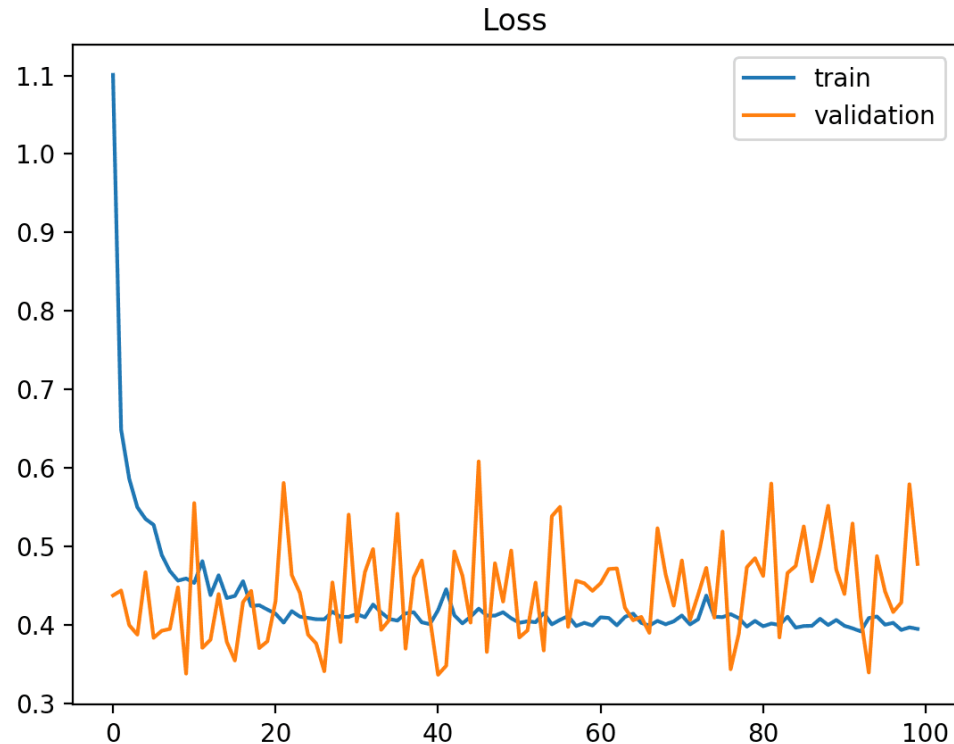


Average Pooling (7x7, stride 7)



Question 16

- What would be your diagnosis/hypothesis if you get the following learning curves?



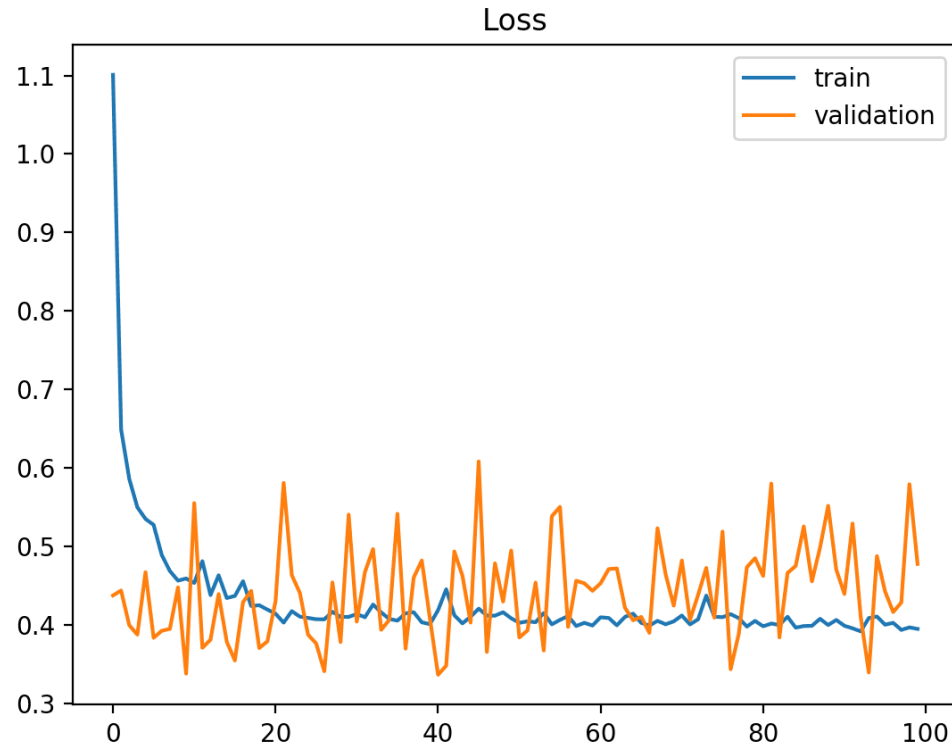
- A. Overfitting scenario: there is a large generalization gap between training and validation.
- B. Inappropriate/Unrepresentative validation set (validation dataset has too few examples).
- C. Underfitting scenario (our model is maybe too simple, and it cannot learn the training set).
- D. Learning rate maybe too large.

Question 16

- What would be your diagnosis/hypothesis if you get the following learning curves?

It's possible to have a small and representative dataset, or a large and unrepresentative one.

So to avoid ambiguities, I removed this line in the PDF I uploaded to PRADO.

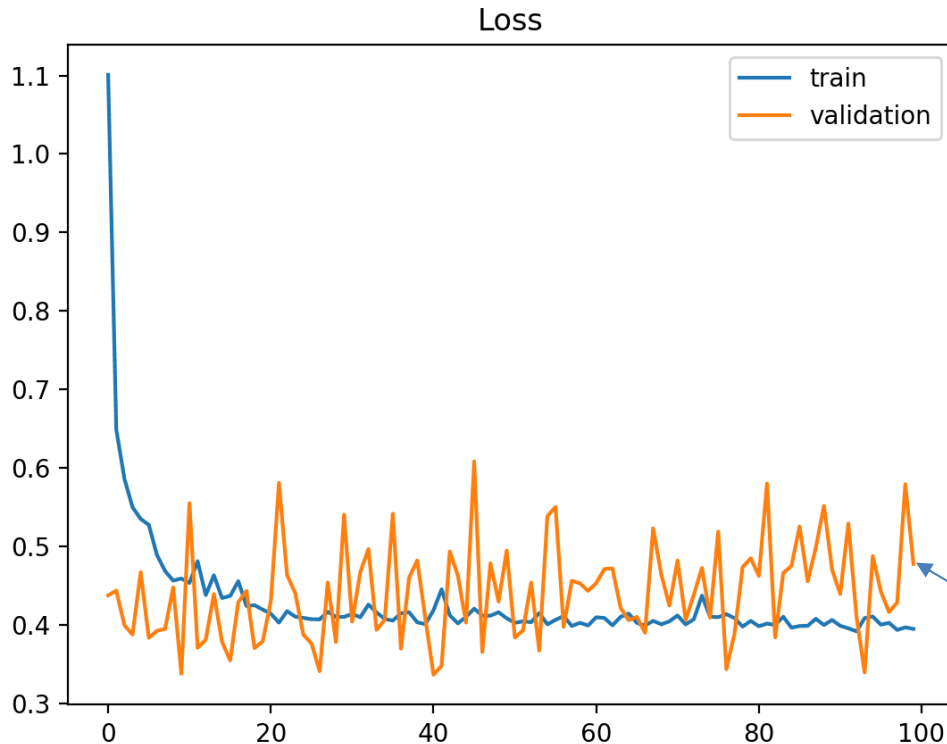


- A. Overfitting scenario: there is a large generalization gap between training and validation.
- B. Inappropriate/Unrepresentative validation set (**validation dataset has too few examples**).
- C. Underfitting scenario (our model is maybe too simple, and it cannot learn the training set).
- D. Learning rate maybe too large.

Question 16

- What would be your diagnosis/hypothesis if you get the following learning curves?

B. Inappropriate/Unrepresentative validation set
(validation dataset has too few examples).



1) To have few examples do not guarantee an unrepresentative validation set, but **increases the “probability” of being unrepresentative**

→ We have less chances of having minority classes represented → less diversity

2) A small validation set usually has higher variance.

This validation performance is completely decorrelated from training

Computing output volumes after convolution

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

You don't need to memorize this mathematical expression.

It is enough that you understand the operation that is carried out when performing a convolution.

Computing output volumes after convolution

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

N=15



Computing output volumes after convolution

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

$N=15$



$P=2 \rightarrow N'=N+2P = 19$



Computing output volumes after convolution

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

N=15



P=2 → N'=N+2P = 19



F=3

What is the max number of movements/displacements we can compute?



Computing output volumes after convolution

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

N=15



P=2 → N'=N+2P = 19



F=3

What is the max number of movements/displacements we can compute?



Computing output volumes after convolution

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

N=15



P=2 → N'=N+2P = 19



F=3

What is the max number of movements/displacements we can compute?



Computing output volumes after convolution

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

N=15



P=2 → N'=N+2P = 19



F=3

What is the max number of movements/displacements we can compute?



Computing output volumes after convolution

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

N=15



P=2 → N'=N+2P = 19



F=3

What is the max number of movements/displacements we can compute?



Computing output volumes after convolution

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

N=15



P=2 → N'=N+2P = 19



F=3

What is the max number of movements/displacements we can compute?



Computing output volumes after convolution

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

N=15

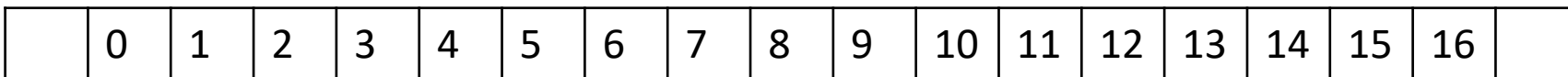


P=2 → N'=N+2P = 19



F=3

What is the max number of movements/displacements we can compute?



Computing output volumes after convolution

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

N=15

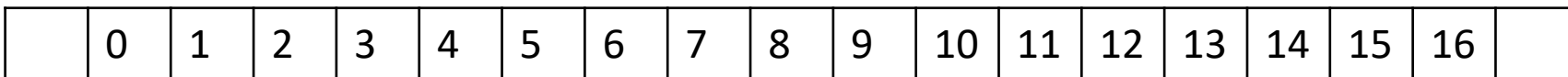


P=2 → N'=N+2P = 19



F=3

What is the max number of movements/displacements we can compute?



N+2P-F = 16



Computing output volumes after convolution

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

$N=15$

$P=2 \rightarrow N'=N+2P = 19$

$F=3$

What is the max number of positions/displacements we can compute?

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	
--	---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	--

$N+2P-F = 16$

This is telling us that, no matter the stride we have, we cannot surpass those 16 displacements!

$0, s, 2s, 3s, 4s, 5s, 6s, \dots, ks \leq 16$, with k an integer

Computing output volumes after convolution

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

N=15

P=2 → N'=N+2P = 19

F=3

What is the max number of positions/displacements we can compute?

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	
--	---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	--

N+2P-F = 16

This is telling us that, no matter the stride we have, we cannot surpass those 16 displacements!

0, s, 2s, 3s, 4s, 5s, 6s, ..., ks ≤ 16, with k an integer

S=3 → k3 ≤ 16 → k ≤ 16/3 → k ≤ 5.3 → k=5 (we need to add 1 to include the first position)

Computing output volumes after convolution

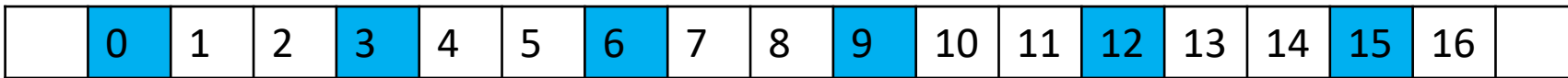
$$\text{Output_size} = ((N + 2 \cdot P - F) / \text{stride}) + 1$$

$N=15$

$P=2 \rightarrow N'=N+2P = 19$

$F=3$

What is the max number of positions/displacements we can compute?



$$N + 2P - F = 16$$

This is telling us that, no matter the stride we have, we cannot surpass those 16 displacements!

$0, s, 2s, 3s, 4s, 5s, 6s, \dots, ks \leq 16$, with k an integer

$S=3 \rightarrow k3 \leq 16 \rightarrow k \leq 16/3 \rightarrow k \leq 5.3 \rightarrow k=5$ (we need to add 1 to include the first position)

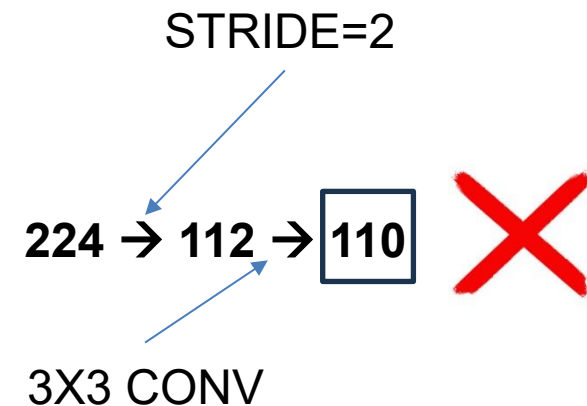
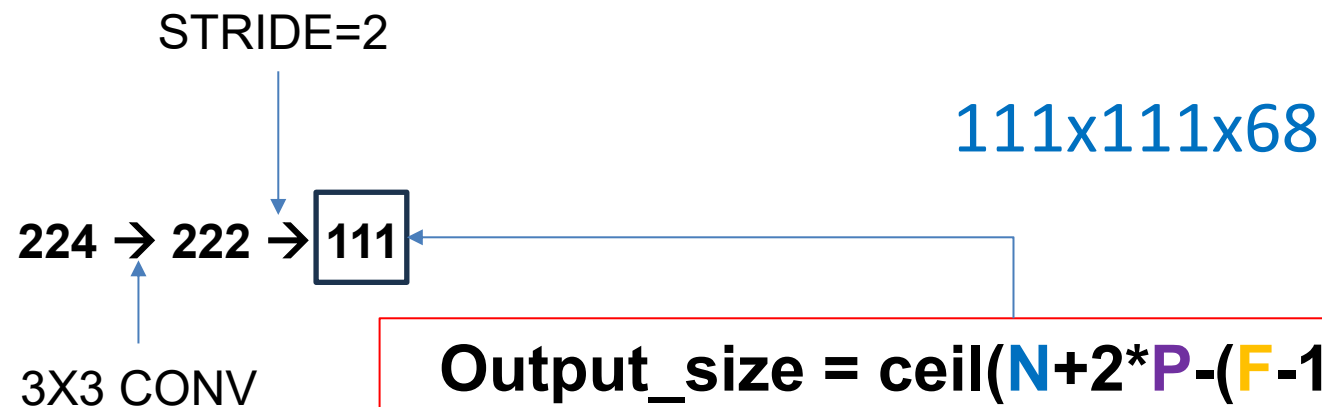
$\rightarrow 0, 3, 6, 9, 12, 15 \leq 16$

Question 20

- Let's assume the following input volume and filters used. What is the resulting output volume?
 - Input volume: 224x224x3
 - Filters used: 68 filters of 3x3 with stride 2 and no padding

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

$$(224 + 2 * 0 - 3) / 2 + 1 = 110.5 + 1 = 111.5 \rightarrow \text{np.floor}(111.5) = 111$$

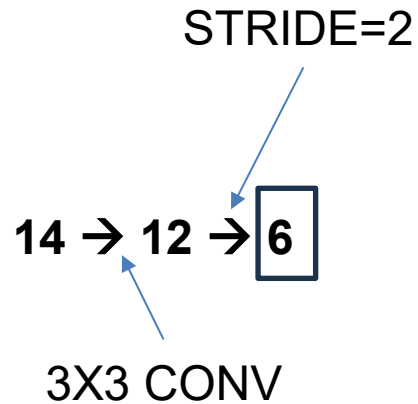


$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

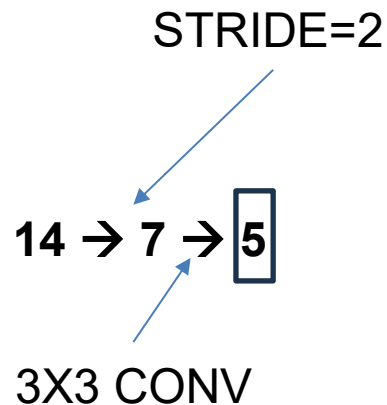
Let's imagine a 14x14 input image, padding=0, stride=2, 3x3 conv

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

Let's imagine a 14x14 input image, padding=0, stride=2, 3x3 conv



Which one of these two strategies is correct?
The first one.



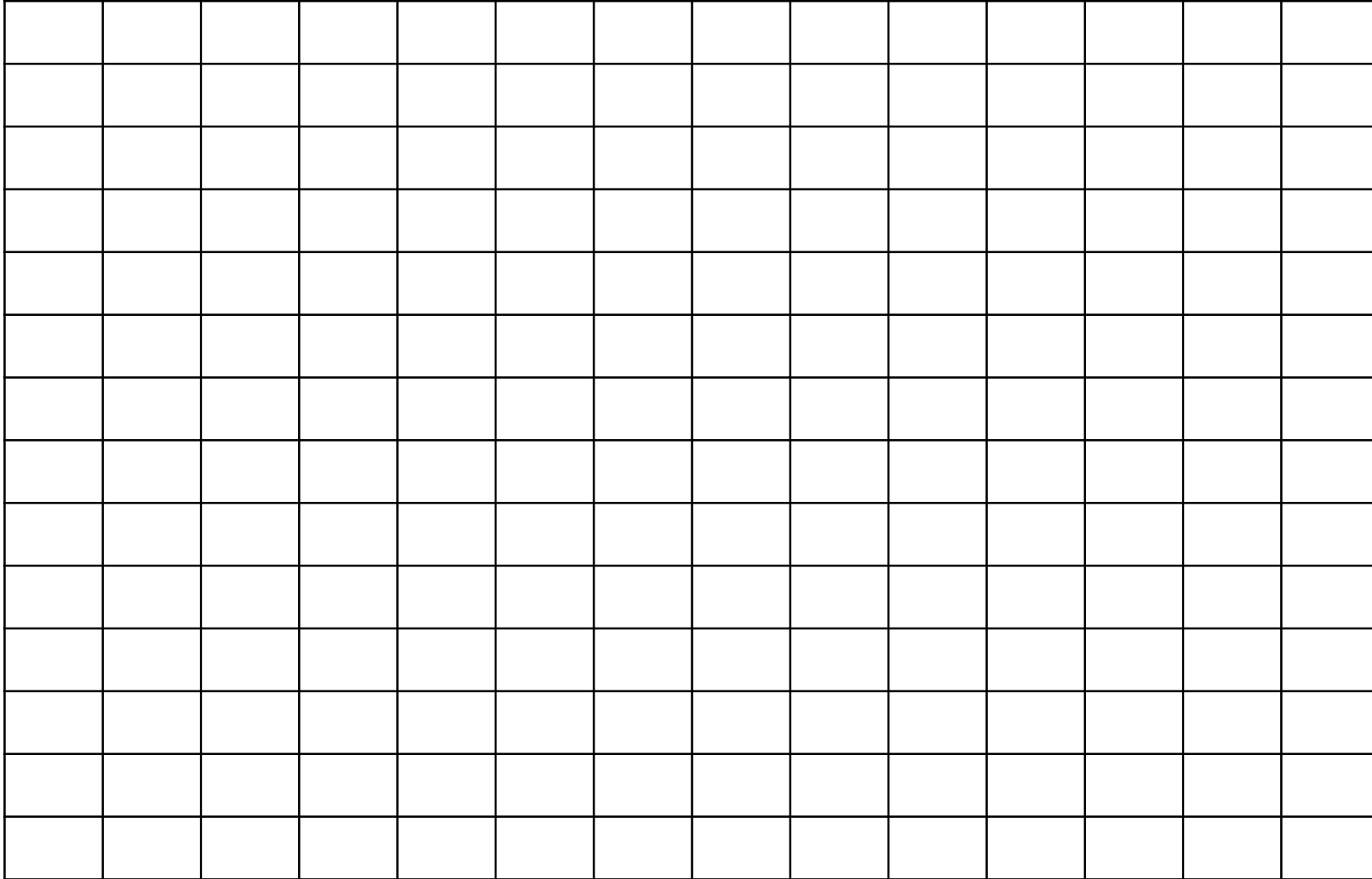
Ok, but why?
Because, otherwise the results are not correct!

Ok, but why we're not dividing prior to the convolution?

We have already seen the explanation, but let's try to find another visual intuition.

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

14x14



Output_size = ((N+2*P-F)/stride) + 1

14x14

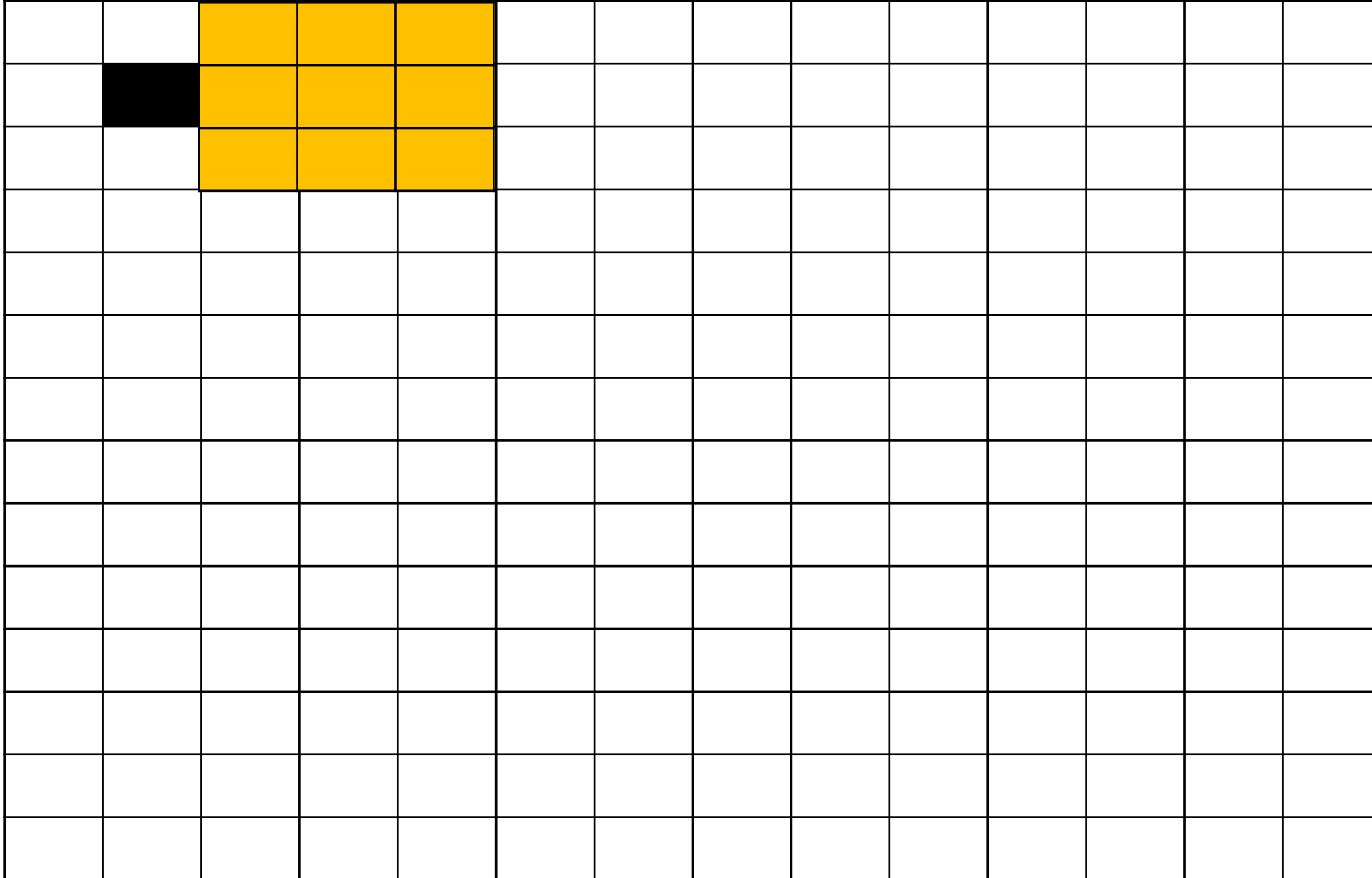
We observe how the convolution actually operates with stride

[illegible]

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

14x14

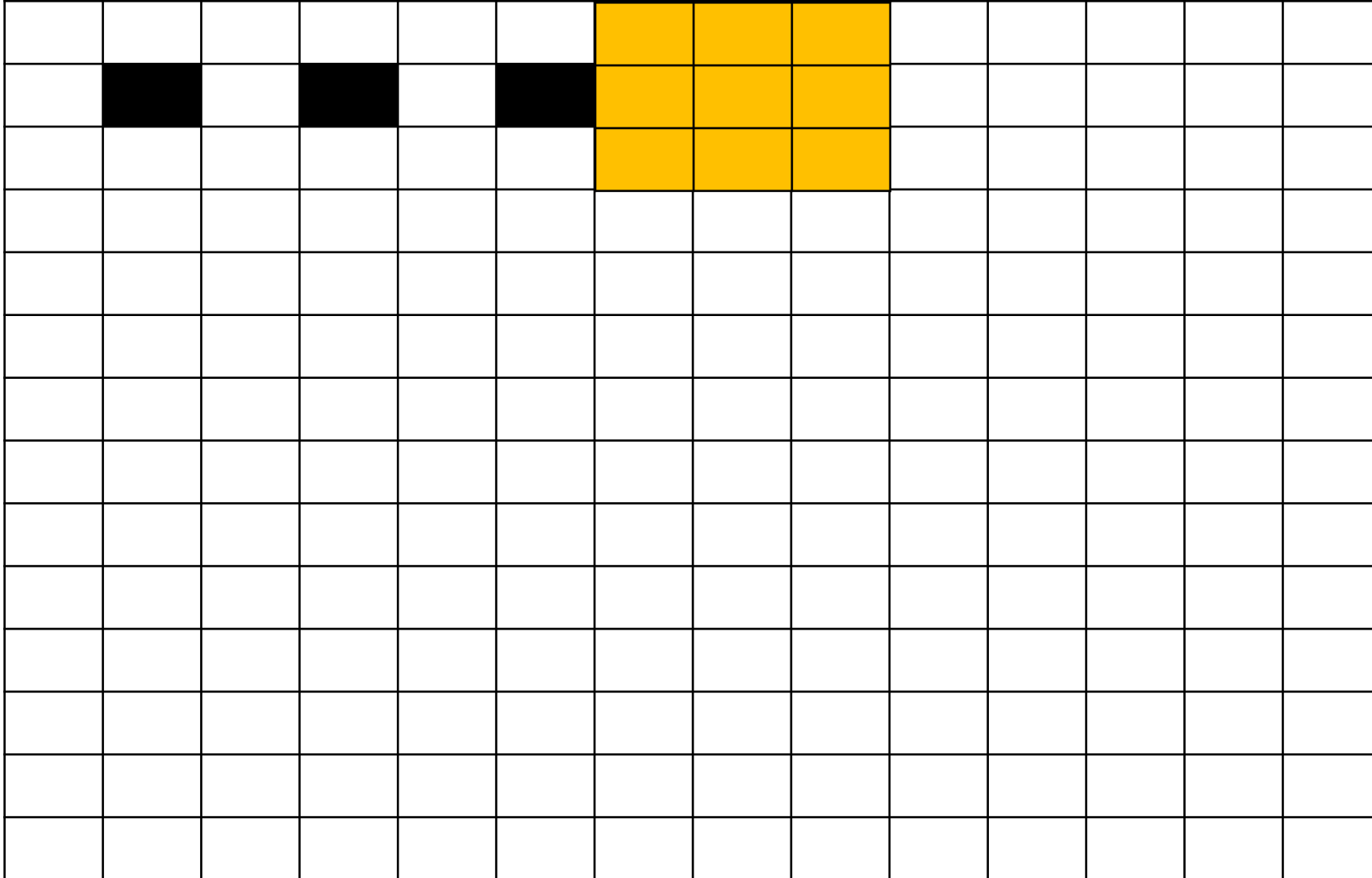
We observe
how the
convolution
actually
operates with
stride



$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

14x14

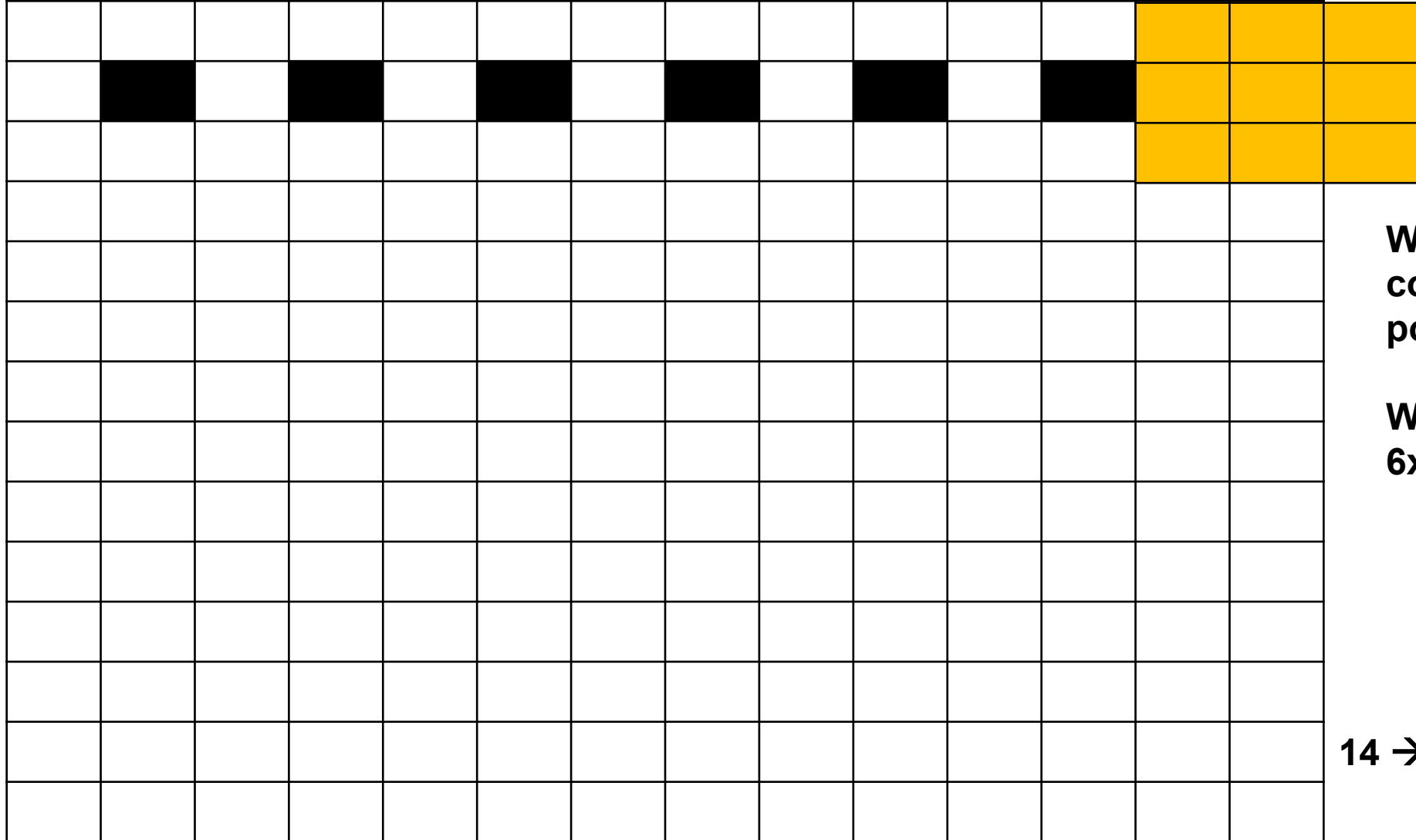
We observe
how the
convolution
actually
operates with
stride



$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

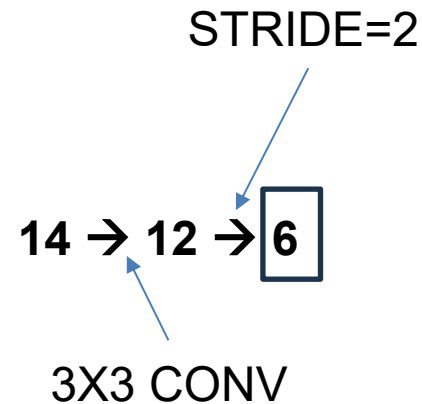
14x14

We observe
how the
convolution
actually
operates with
stride



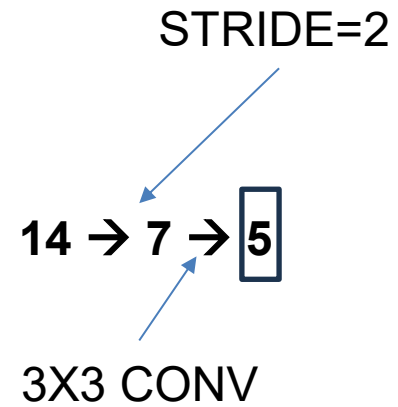
We cannot
compute this
position!

We have a
6x6 output



$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

What would happen if we divide first?



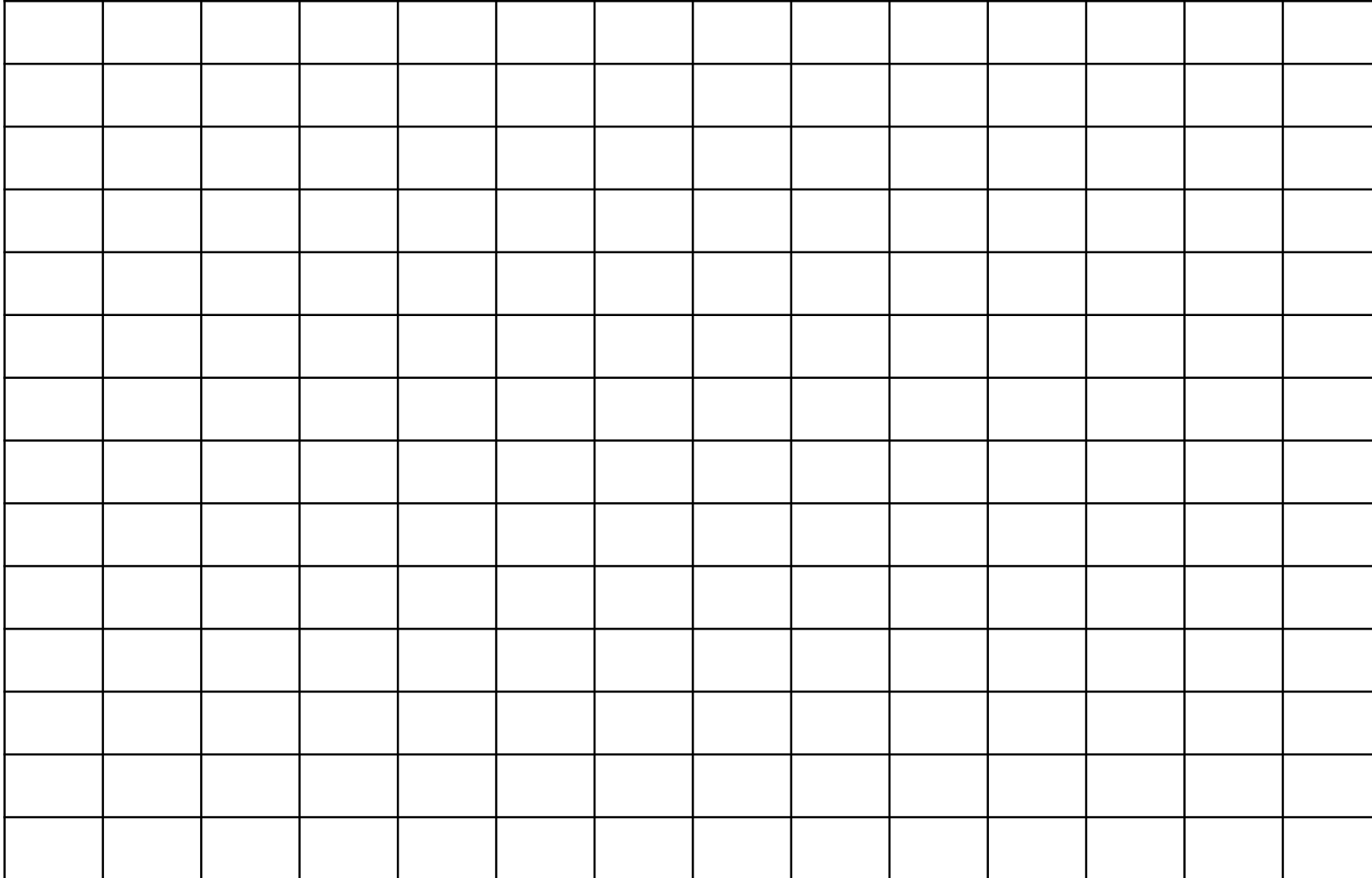
$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

14x14

We divide first
and apply the
3x3 kernel
afterwards



we're first
performing
downsampling
and then a
normal
convolution



$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

14x14

**We divide first
and apply the
3x3 kernel
afterwards**



**we're first
performing
downsampling
and then a
normal
convolution**

[illegible]

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

14x14

**We divide first
and apply the
3x3 kernel
afterwards**



**we're first
performing
downsampling
and then a
normal
convolution**

[illegible]

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

14x14

**We divide first
and apply the
3x3 kernel
afterwards**



**we're first
performing
downsampling
and then a
normal
convolution**

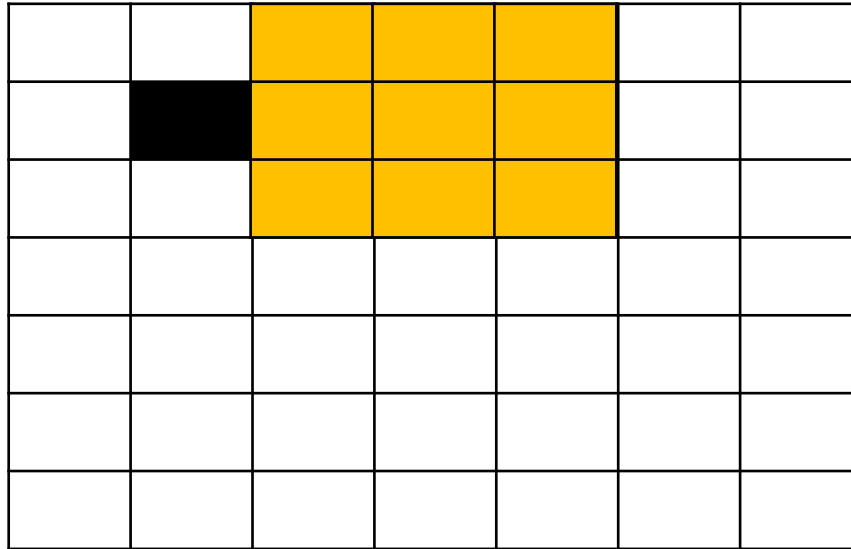
$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

14x14

**We divide first
and apply the
3x3 kernel
afterwards**



**we're first
performing
downsampling
and then a
normal
convolution**



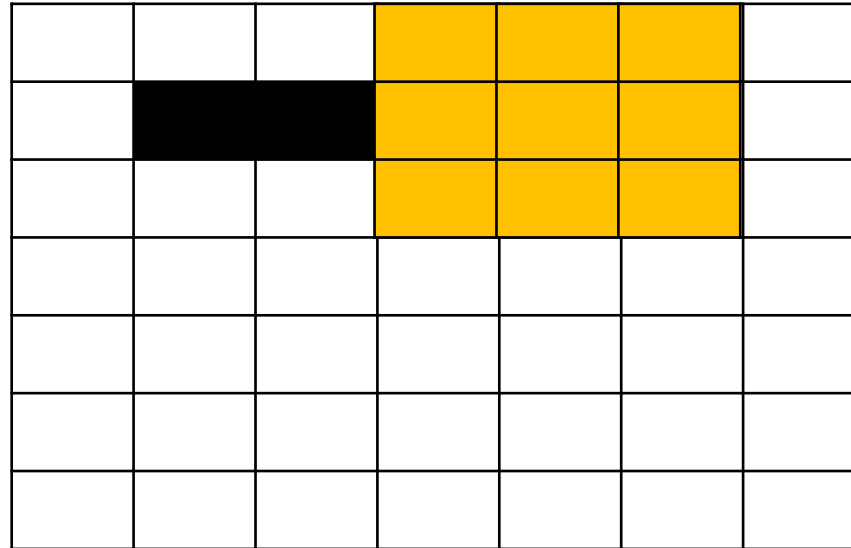
$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

14x14

We divide first
and apply the
3x3 kernel
afterwards



we're first
performing
downsampling
and then a
normal
convolution



$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

14x14

**We divide first
and apply the
3x3 kernel
afterwards**

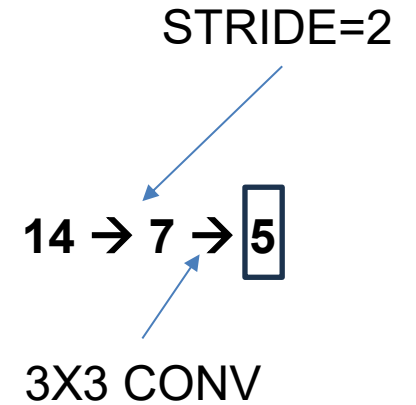
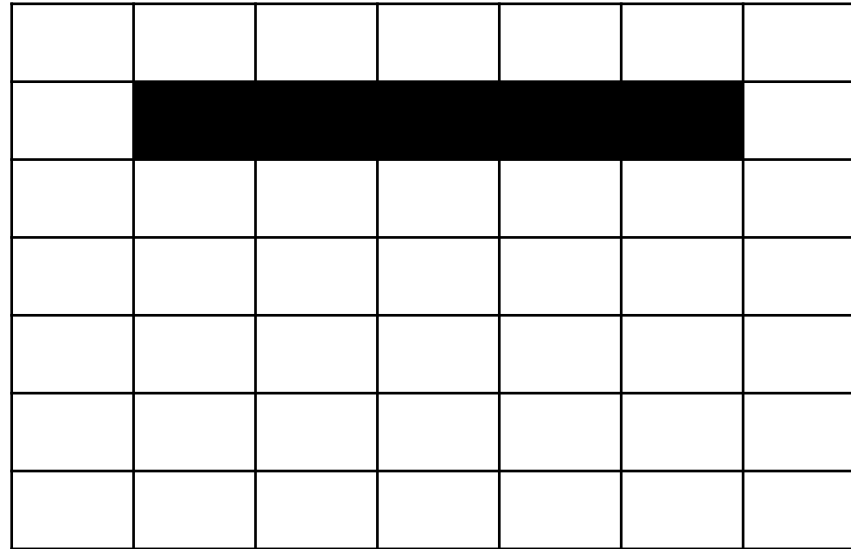


**we're first
performing
downsampling
and then a
normal
convolution**

$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

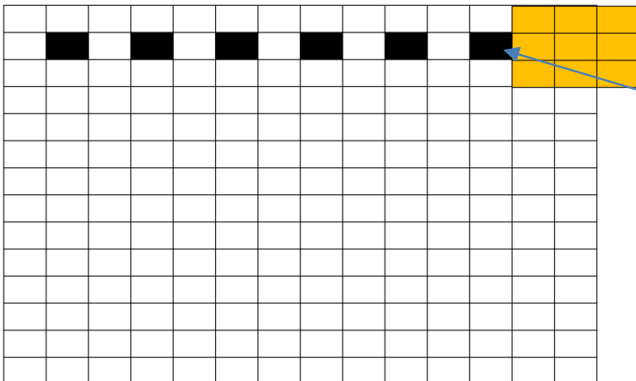
14x14

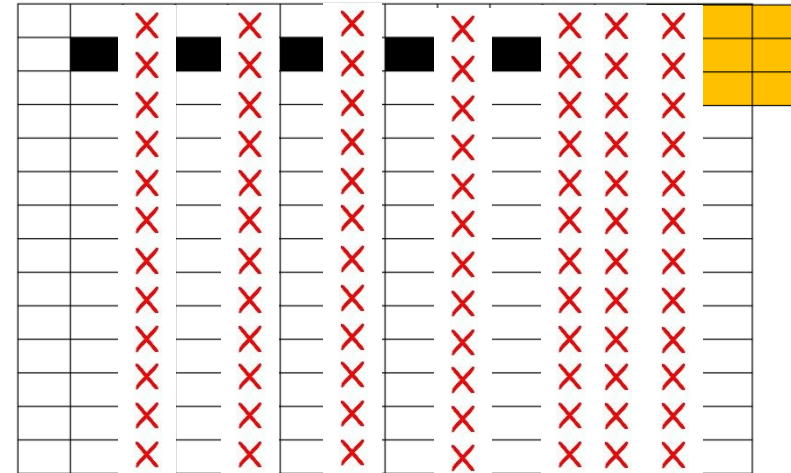
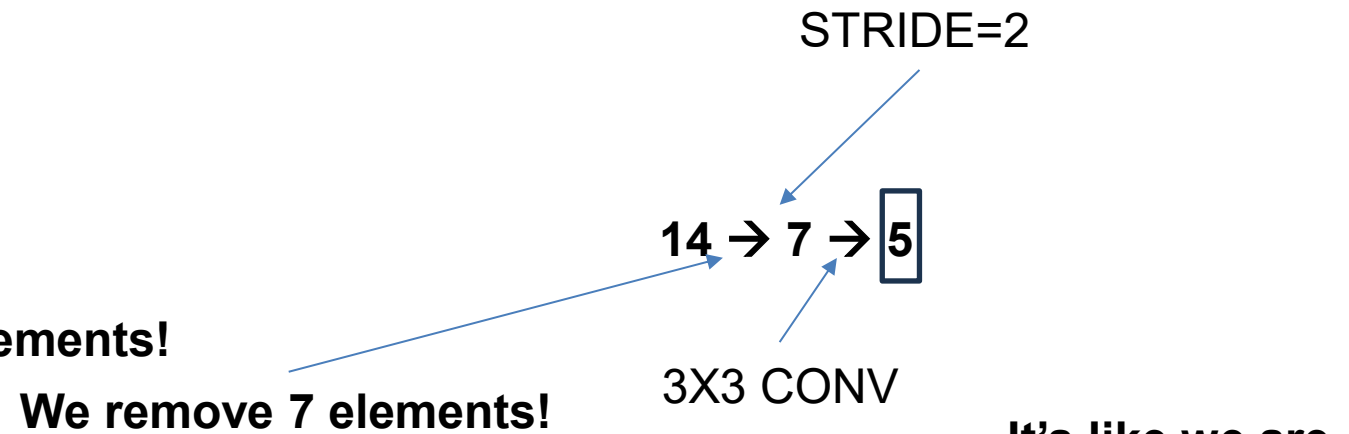
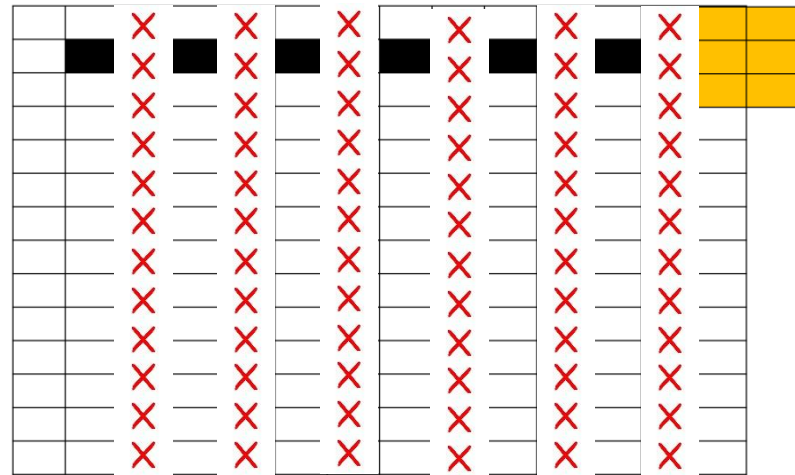
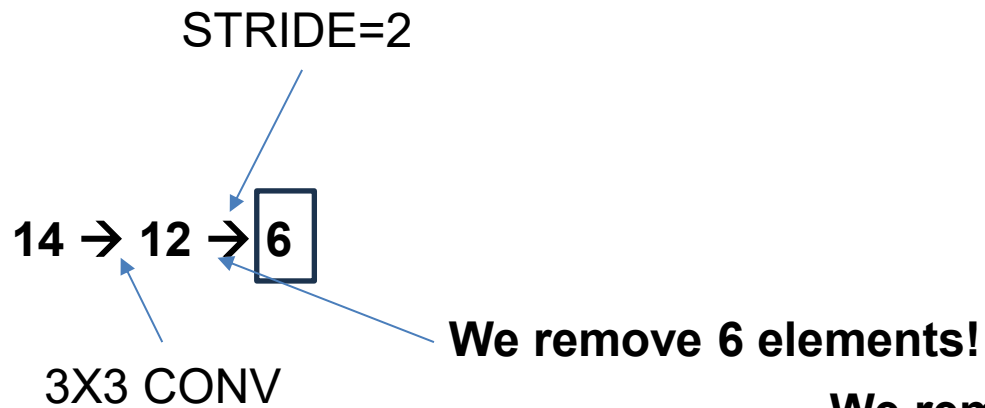
We divide first
and apply the
3x3 kernel
afterwards



In this case, dividing first by the stride, **we're removing a pixel that we need to compute the last value.**

We can't compute this last value because we're missing a pixel: **if we remove 7 pixels (instead of 6), we can compute just 5 output values.**





CONCLUSION:

We want to compute the correct size after a convolution.

Use the “strategy” you consider clearer and more intuitive, but try to verify your results manually.

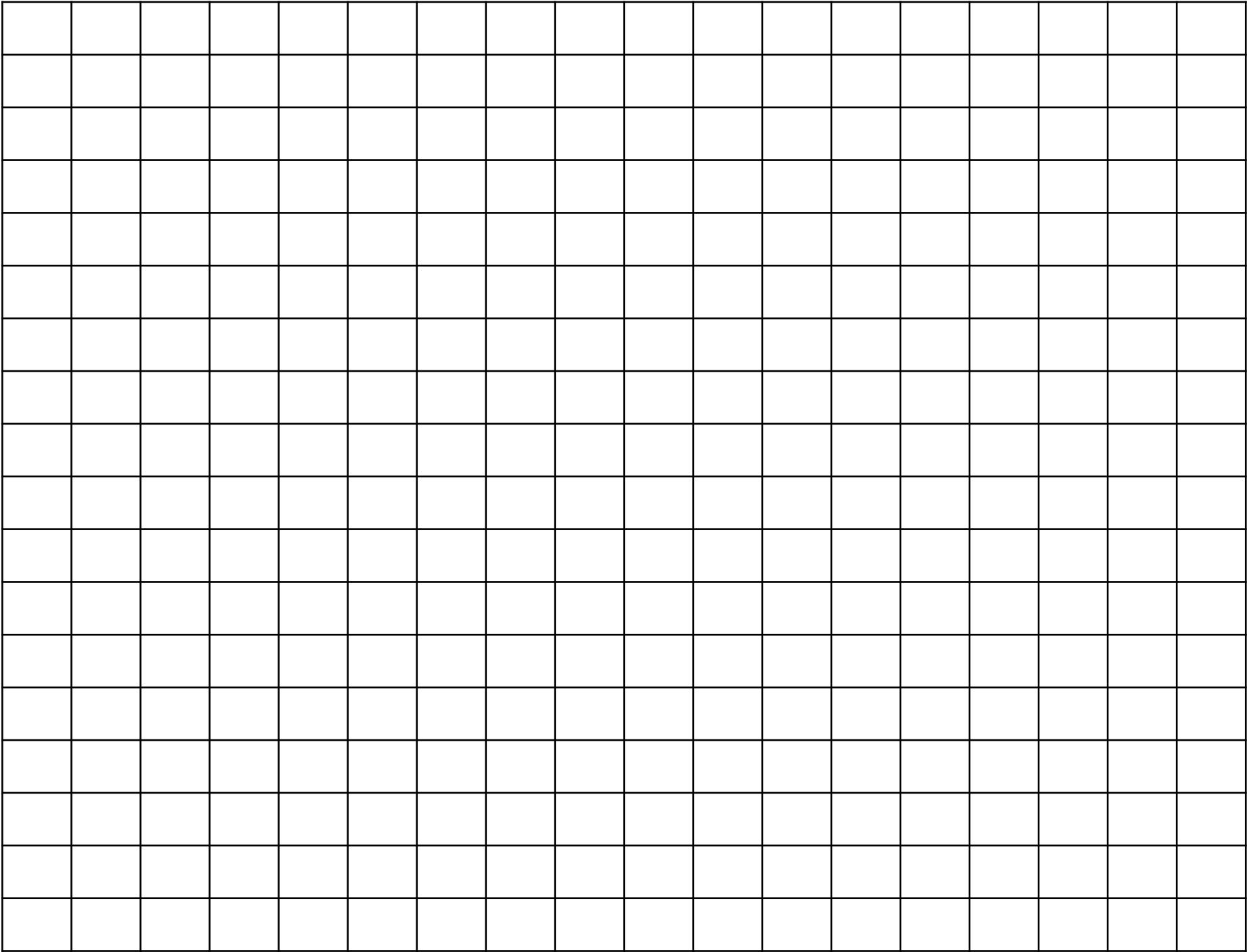
Exam 2024-25

14x14

7x7
padding=2
stride=3

14x14

7x7
padding=2
stride=3



[illegible]

padding=2

stride=3

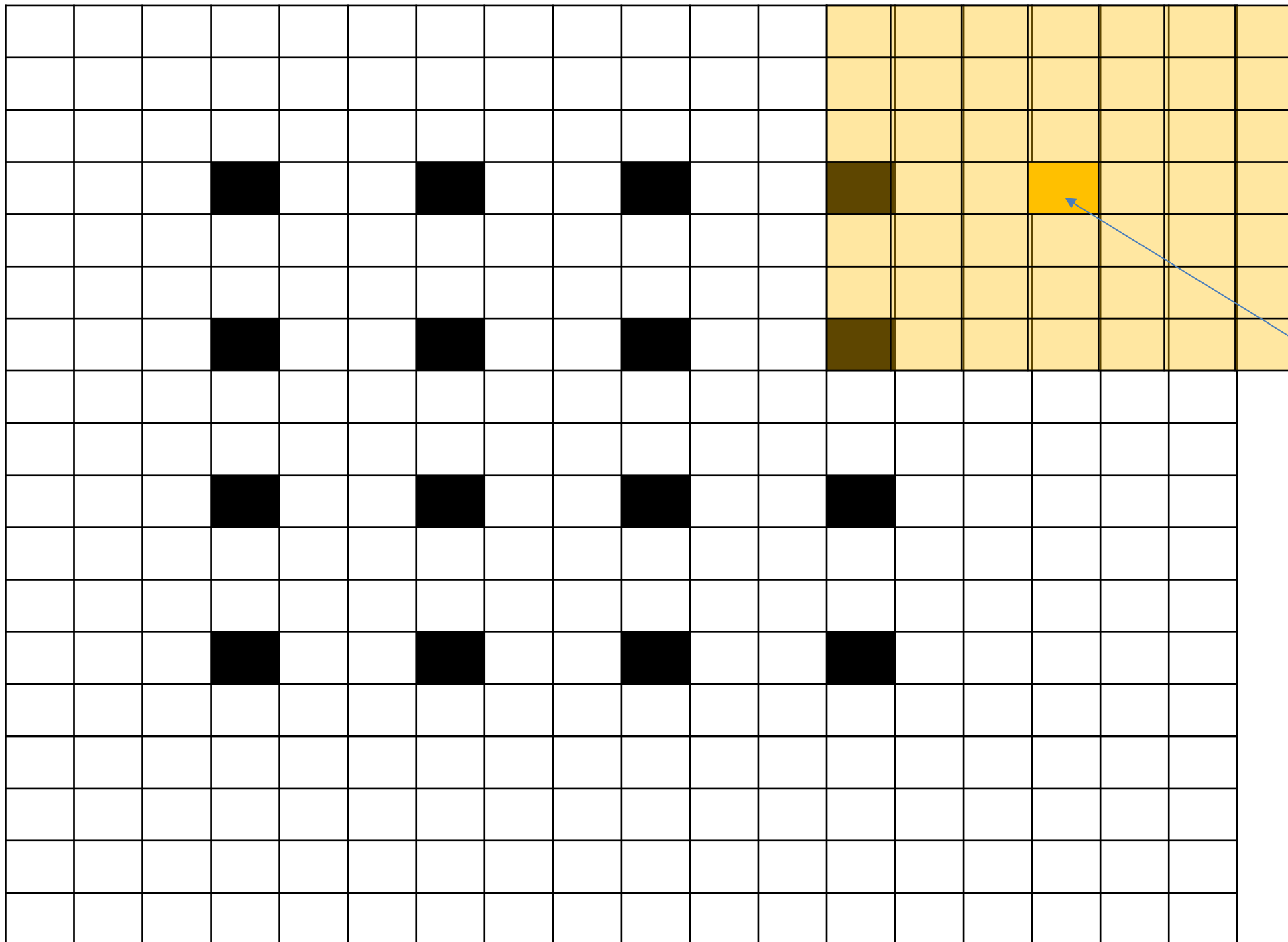
[illegible]

padding=2

stride=3

14x14

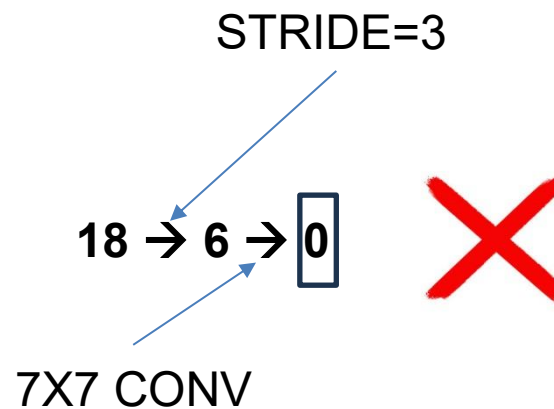
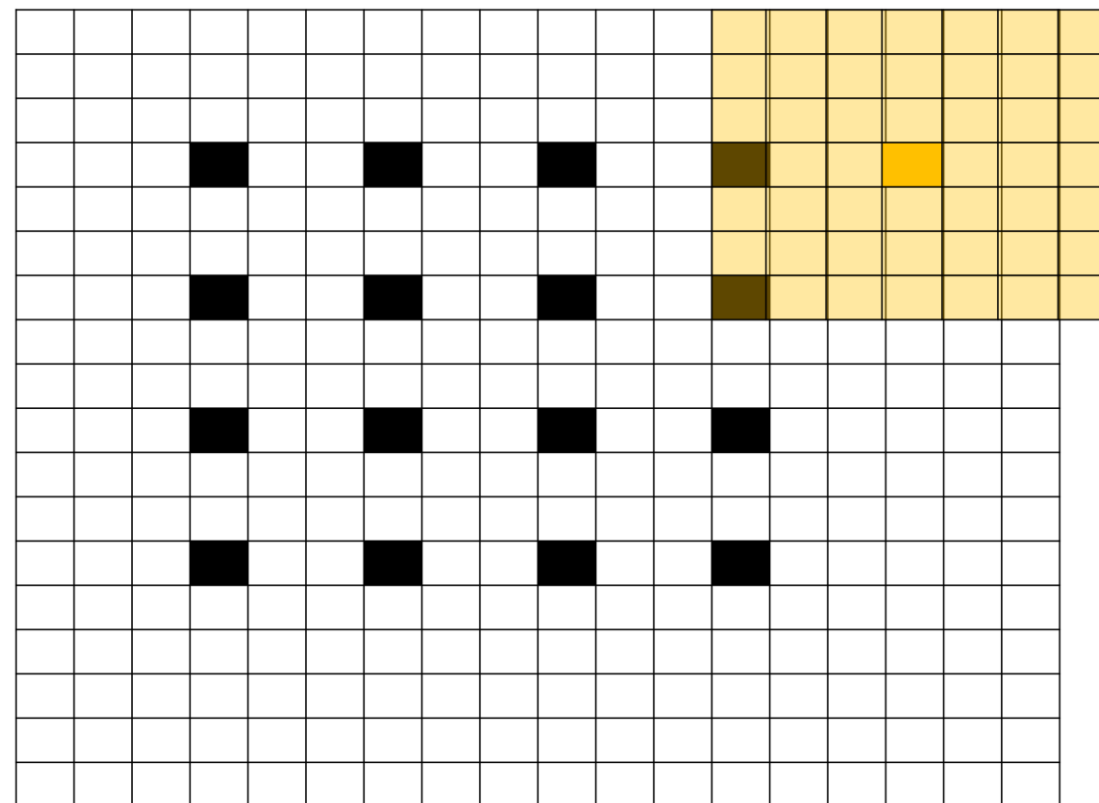
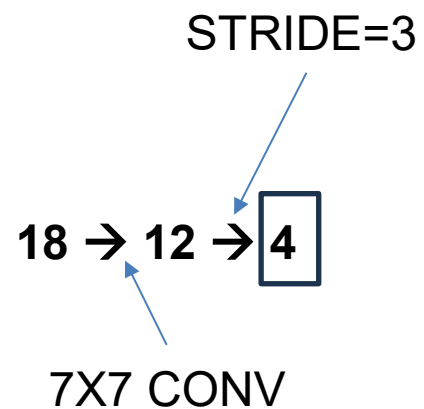
7x7
padding=2
stride=3



**We can't
compute
this value!**

14x14

7x7
padding=2
stride=3

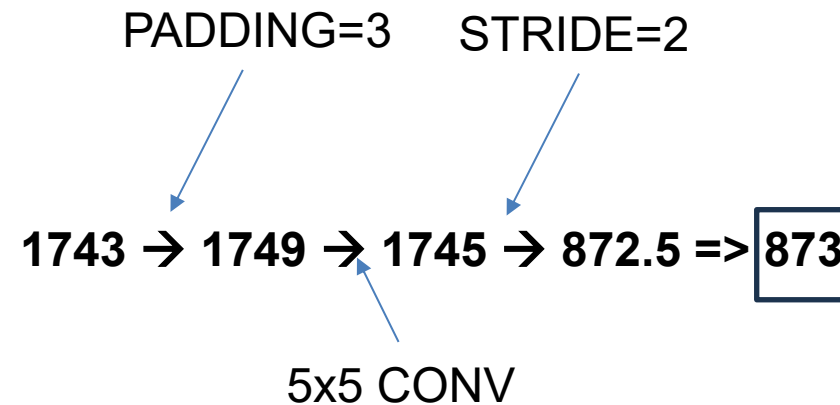


Let's face another example:

1743x1743 input image, 5x5 filter, padding=3, stride=2

Let's face another example:

1743x1743 input image, 5x5 filter, padding=3, stride=2



$$\begin{aligned}\text{Output_size} &= ((N+2*P-F)/\text{stride}) + 1 = ((1743+2*3-5)/2) + 1 = ((1749-5)/2) + 1 \\ &= ((1744)/2) + 1 = 872+1 = 873\end{aligned}$$